NAME: V Vishwa Teja ID NO: 2520090033

SKILL WEEK 7

1. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.7.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command < input_file\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *redirect = strchr(command, '<');

    if (redirect == NULL)
    {
        printf("Input redirection not found\n");
        return 1;
    }

    *redirect = '\0';

    char *input_file = redirect + 1;

    while (*input_file == ' ')
        input_file++;

    char *end = input_file + strlen(input_file) - 1;

    while (end > input_file && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
    }

    int fd = open(input_file, O_RDONLY);

    if (fd < 0)
```

```
{
    perror("Input file");
    return 1;
}

char *cmd = command;

while (*cmd == ' ')
    cmd++;

char *args[64];
int count = 0;

char *token = strtok(cmd, " \t\n");

while (token != NULL && count < 63)
{
    args[count++] = token;
    token = strtok(NULL, " \t\n");
}

args[count] = NULL;

if (count == 0)
{
    printf("No command specified\n");
    close(fd);
    return 1;
}

int saved_stdin = dup(STDIN_FILENO);

if (dup2(fd, STDIN_FILENO) < 0)
{
    perror("dup2");
    close(fd);
    return 1;
}

close(fd);

pid_t pid = fork();

if (pid == 0)
```

```
    {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        dup2(saved_stdin, STDIN_FILENO);
        close(saved_stdin);
        return 1;
    }

    waitpid(pid, NULL, 0);

    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);

    printf("Input redirection completed successfully.\n");

    return 0;
}
```

Create a input file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "Hello from Skill Week 7" > input.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat input.txt
Hello from Skill Week 7
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test input redirection:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.1 "cat < input.txt"
Hello from Skill Week 7
Input redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test with wc:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.1 "wc < input.txt"
      1       5      24
Input redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test missing file handling:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.1 "cat < missing.txt"
Input file: No such file or directory
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

2. To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.7.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command > output_file\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *redirect = strchr(command, '>');

    if (redirect == NULL)
    {
        printf("Output redirection not found\n");
        return 1;
    }

    *redirect = '\0';

    char *output_file = redirect + 1;

    while (*output_file == ' ')
        output_file++;

    char *end = output_file + strlen(output_file) - 1;

    while (end > output_file && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
    }

    int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
```

```
{
    perror("Output file");
    return 1;
}

char *cmd = command;

while (*cmd == ' ')
    cmd++;

char *args[64];
int count = 0;

char *token = strtok(cmd, " \t\n");

while (token != NULL && count < 63)
{
    args[count++] = token;
    token = strtok(NULL, " \t\n");
}

args[count] = NULL;

if (count == 0)
{
    printf("No command specified\n");
    close(fd);
    return 1;
}

int saved_stdout = dup(STDOUT_FILENO);

if (dup2(fd, STDOUT_FILENO) < 0)
{
    perror("dup2");
    close(fd);
    return 1;
}

close(fd);

pid_t pid = fork();

if (pid == 0)
```

```
    {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
        return 1;
    }

    waitpid(pid, NULL, 0);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("Output redirection completed successfully.\n");

    return 0;
}
```

Test output redirection:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.7.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill.7.2.c -o skill.7.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.2 "echo Hello from Skill Week 7 > output.txt"
Output redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat output.txt
Hello from Skill Week 7
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Verify output file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ls -l output.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 24 Oct  6 15:38 output.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ stat output.txt
  File: output.txt
  size: 24              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 52710       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vishwa_teja_veerandi)   Gid: ( 1000/vishwa_teja_veerandi)
Access: 2026-10-06 15:38:36.777109742 +0000
Modify: 2026-10-06 15:38:28.385110186 +0000
Change: 2026-10-06 15:38:28.385110186 +0000
 Birth: 2026-10-06 14:09:08.620976957 +0000
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test permission handling:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ mkdir protected
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ chmod 555 protected
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.2 "echo Test > protected/output.txt"
Output file: Permission denied
```

3. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.7.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command >> output_file\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *redirect = strstr(command, ">>");

    if (redirect == NULL)
    {
        printf("Append redirection not found\n");
        return 1;
    }

    *redirect = '\0';

    char *output_file = redirect + 2;

    while (*output_file == ' ')
        output_file++;

    char *end = output_file + strlen(output_file) - 1;

    while (end > output_file && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
    }

    int fd = open(output_file, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd < 0)
    {
        perror("Output file");
        return 1;
    }
```

```c
    char *cmd = command;

    while (*cmd == ' ')
        cmd++;

    char *args[64];
    int count = 0;

    char *token = strtok(cmd, " \t\n");

    while (token != NULL && count < 63)
    {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    if (count == 0)
    {
        printf("No command specified\n");
        close(fd);
        return 1;
    }

    int saved_stdout = dup(STDOUT_FILENO);

    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    pid_t pid = fork();

    if (pid == 0)
    {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
```

```
        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
        return 1;
    }

    waitpid(pid, NULL, 0);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("Append redirection completed successfully.\n");

    return 0;
}
```

Create a initial data:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "First line" > append.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat append.txt
First line
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Append the data:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.3 "echo Second line >> append.txt"
Append redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat append.txt
First line
Second line
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.3 "echo Third line >> append.txt"
Append redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat append.txt
First line
Second line
Third line
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Verify file size:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ls -l append.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 34 Oct  7 02:58 append.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ stat append.txt
  File: append.txt
  size: 34              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 52728       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vishwa_teja_veerandi)   Gid: ( 1000/vishwa_teja_veerandi)
Access: 2026-10-07 02:58:46.710568604 +0000
Modify: 2026-10-07 02:58:40.930569240 +0000
Change: 2026-10-07 02:58:40.930569240 +0000
 Birth: 2026-10-07 02:57:36.386574949 +0000
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.7.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command 2> error_file\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *redirect = strstr(command, "2>");

    if (redirect == NULL)
    {
        printf("stderr redirection not found\n");
        return 1;
    }

    *redirect = '\0';

    char *error_file = redirect + 2;

    while (*error_file == ' ')
        error_file++;

    char *end = error_file + strlen(error_file) - 1;

    while (end > error_file && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
    }

    int fd = open(error_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("Error file");
        return 1;
    }
```

```c
    char *cmd = command;

    while (*cmd == ' ')
        cmd++;

    char *args[64];
    int count = 0;

    char *token = strtok(cmd, " \t\n");

    while (token != NULL && count < 63)
    {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    if (count == 0)
    {
        printf("No command specified\n");
        close(fd);
        return 1;
    }

    int saved_stderr = dup(STDERR_FILENO);

    if (dup2(fd, STDERR_FILENO) < 0)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    pid_t pid = fork();

    if (pid == 0)
    {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
```

```c
        dup2(saved_stderr, STDERR_FILENO);
        close(saved_stderr);
        return 1;
    }

    waitpid(pid, NULL, 0);

    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stderr);

    printf("stderr redirection completed successfully.\n");

    return 0;
}
```

Test an error:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.4 "ls missing_folder 2> error.txt"
stderr redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat error.txt
ls: cannot access 'missing_folder': No such file or directory
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test successful command:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.4 "ls 2> error.txt"
Argument-Passing     largefile.bin    practical12.2              practical4.2.c          practical8.1.c          protected     skill1.2    skill3          skill4.1.c  skill5.4.c
File2.txt            linux            practical12.2.c            practical5.1            practical8.2            sample.txt    skill1.2.c  skill3.1        skill4.2    skill6.1
Makefile             mmap_source.txt  practical12.2.prevention    practical5.1.c          practical8.2.c          sample2.txt   skill1.3    skill3.1.c      skill4.2.c  skill6.1.c
append.txt           myprogram        practical12.2.prevention.c practical5.2            practical9.1            server_fifo   skill1.3.c  skill3.2        skill4.3    skill6.2
argument_passing     output.txt       practical2.1               practical5.2.c          practical9.1.c          skill.7.1     skill2      skill3.2.c      skill4.3.c  skill6.2.c
argument_passing.c   practical10      practical2.1.c             practical6.1            practical9.1.standard   skill.7.1.c   skill2.1    skill3.3        skill4.4    source.txt
client_fifo          practical10.2    practical2.1.txt           practical6.1.c          practical9.1.standard.c skill.7.2     skill2.1.c  skill3.3.c      skill4.4.c  sum1
copy.txt             practical10.2.c  practical3.1               practical6.1_client     practical9.2            skill.7.2.c   skill2.2    skill3.4        skill5.1    sum1.c
copy_mmap.txt        practical11.1    practical3.1.c             practical6.1_client.c   practical9.2.c          skill.7.3     skill2.2.c  skill3.4.c      skill5.1.c  task2
copy_read_write.txt  practical11.1.c  practical3.2               practical6.2            practical_fixed         skill.7.3.c   skill2.3    skill3.c        skill5.2    task2.c
destination.txt      practical11.2    practical3.2.c             practical6.2.c          practical_fixed.c       skill.7.4     skill2.3.c  skill3main2     skill5.2.c  teja
error.txt            practical11.2.c  practical4.1               practical7.1            problem1                skill.7.4.c   skill2.4    skill3main2.c   skill5.3    testfile.bin
files.txt            practical12.1    practical4.1.c             practical7.1.c          problem1.c              skill1.1      skill2.4.c  skill3main2.txt skill5.3.c  vishwa
input.txt            practical12.1.c  practical4.2               practical8.1            problem1.txt            skill1.1.c    skill2.c    skill4.1        skill5.4    vishwa.c
stderr redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat error.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Test another error:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.7.4 "cat nonexistent.txt 2> errors.txt"
stderr redirection completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat errors.txt
cat: nonexistent.txt: No such file or directory
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```